

GRAPH-GUIDED LONG-HORIZON MPC WITH WORLD MODELS

Anonymous authors

Paper under double-blind review

ABSTRACT

The cross-entropy method (CEM) planner used with LeWorldModel (LeWM) ranks candidate action sequences by the Euclidean distance between a predicted terminal latent and a goal latent. While this objective succeeds on short-horizon goals, its performance degrades sharply as goals become more temporally distant or come from a different episode. On Push-T, the planner reaches same-episode goals 25, 50 and 100 steps ahead, and cross-episode goals, with success rates of 91/41/13/14%, respectively. We address this degradation at long horizons and across episodes without retraining the world model or learning a subgoal generator. We learn a temporal distance representation over frozen latents, build a graph of the offline dataset, and select a subgoal along the graph’s shortest path. A reachability critic scores predicted endpoints by expected time to goal, and latent L2 is used for the final horizon. Each component addresses a distinct limitation: the subgoal provides *direction*, the critic accounts for *reachability*, and the switch to L2 restores near-goal *resolution*. The graph finds a route for every evaluated cross-episode pair, but reaching the goal still requires the critic. Across goals at offsets 50 and 100, and on cross-episode goals, our method raises success from 41/13/14% to 70/46/42% on Push-T, from 91/83/29% to 98/98/66% on Reacher, and from 34/36/16% to 50/50/28% on Cube. At the 25-step horizon, our method trails L2 by 2.8 points on Push-T (88.4 vs 91.2) and 1.2 points on Reacher (76.0 vs 77.2), but leads by 6.0 points on Cube (62.0 vs 56.0).

1 INTRODUCTION

Joint-embedding world models trained from pixels (Maes et al., 2026; Zhou et al., 2024; Sobal et al., 2025) plan by search: they sample candidate action sequences, roll each through a learned latent predictor, and select the sequence whose predicted terminal latent lies closest to the goal latent in Euclidean (L2) distance – the default objective we call L2 throughout. On the horizons these models are typically evaluated on, where the goal is a frame from the same trajectory a fixed number of steps ahead, this objective performs well. Its performance degrades sharply, however, as the goal becomes more temporally distant. With the frozen LeWorldModel (LeWM) checkpoint and its published cross-entropy-method (CEM) planner, Push-T success falls from 91% for goals 25 steps ahead to 13% at 100 steps and 14% for goals drawn from a different episode. The failure is not a budget constraint: a 250-step budget instead of 100 adds barely two points (table 3b). In section 5 we show that it lies in the objective itself, not just in the predictor.

A substantial body of recent work targets the same failure in the same frozen-model setting, almost uniformly by learning something new on top of it: a subgoal generator and action prior (Cheng et al., 2026), a macro-action space and high-level predictor (Caselli et al., 2026), world models operating at several time scales (Zhang et al., 2026), an action-free latent planner (Masip et al., 2026), or a retrained encoder whose geometry directly encodes temporal cost (Bai & Xiong, 2026; Li et al., 2026b;a). A separate, older line of work in offline goal-conditioned reinforcement learning addresses long horizons by searching over the data directly: it builds a graph over the offline dataset and lets shortest paths select subgoals (Savinov et al., 2018; Eysenbach et al., 2019; Emmons et al., 2020; Baek et al., 2025; Opryshko et al., 2025; ALPS authors, 2026), handing the resulting subgoal to a learned goal-conditioned policy. To our knowledge, no prior method places the dataset graph inside a pixel world model’s planner itself. Caselli et al. (2026) point to why that would help: from their own hierarchical planner, they identify the conditions under which a subgoal interface transfers

to a compact frozen world model – subgoals must remain “data-supported, executable, and matched to the temporal scale.”

We place the dataset graph inside LeWM’s planner: a subgoal that is data-supported, executable, and matched to the temporal scale by construction. We learn a temporal distance representation (TDR) on the frozen LeWM latents (Park et al., 2024; Baek et al., 2025), build a graph over the offline dataset in that space, and replace LeWM’s terminal cost with the distance to a subgoal – a dataset frame reached from the current state by the shortest path of observed transitions toward the goal, truncated to one planning horizon. This alone meaningfully improves the mid-range regime, but does nothing for cross-episode goals: the graph finds a route for every pair we evaluate, well inside the planning budget, yet the planner does not execute it. The missing component is reachability, which we address with a critic $V(z, z_g, h)$, the probability that the goal is reachable from latent z within h steps, and score each predicted endpoint by *expected hitting time* under the steps remaining. With this term, and with the final horizon handed back to LeWM’s own objective once the goal is within one horizon, a single configuration matches or exceeds every alternative we tested at offsets 50 and above and on cross-episode goals, across three environments – Push-T, Reacher, and Cube (table 1).

Our contributions are:

1. A planner-side method for long-horizon and cross-episode goal reaching with a frozen pixel world model, requiring no new generator, policy, or predictor: a TDR on frozen latents, a dataset graph, and a hitting-time critic, all scored through LeWM’s own CEM planner without modification (section 3).
2. A budget-clocked expected-hitting-time cost that replaces the **standard** – $\log V$ feasibility term, removes its dependence on a weighting hyperparameter, and preserves a progress gradient among feasible candidates (sections 3.3 and 4.3).
3. **The first** cross-episode goal-reaching evaluation for latent MPC on Push-T, Reacher, and Cube (section 4.1 and table 1).
4. A **mechanistic account**, supported by per-pair diagnostics, of which failure mode each component addresses: direction, reachability, and resolution (section 4.4).

2 RELATED WORK

Planning with latent world models. PlaNet and Dreamer (Hafner et al., 2019; 2023) plan or learn policies in a reconstruction-based latent; DINO-WM (Zhou et al., 2024), PLDM (Sobal et al., 2025), V-JEPA 2-AC (Assran et al., 2025) and LeWM (Maes et al., 2026) plan with zero-shot CEM in a joint-embedding latent, scoring candidates by terminal latent distance. Terver et al. (2025) and stable-worldmodel authors (2026) study which design choices matter; the planning horizon is a known weak point of all of them.

Repairing the cost on a frozen model. Trajectory reachability metrics (Li et al., 2026a) learn a pairwise temporal cost from logged trajectories and use it as the terminal ranking; temporal-distance JEPA (Bai & Xiong, 2026), RC-aux (Li et al., 2026b), DA-LeWM (Wang et al., 2026) and SCALE (Hu et al., 2026) push similar signals into the encoder at training time, and Destrade et al. (2026); Temporal straightening authors (2026) shape the geometry toward a value or a straight path. Our TDR is in the same family (an expectile temporal distance on frozen latents) but we do not use it as the cost: we use it to build a graph and pick a data-supported target, and we show the metric alone is not what helps (section 4.3). RC-aux’s budget-conditioned reachability is the closest relative of our critic; ours is a separate network with an explicit plan clock and never touches the encoder.

Hierarchical planning with world models. HWM (Zhang et al., 2026) learns world models at several temporal scales and uses long-horizon predictions as subgoals; SAGE (Cheng et al., 2026) learns a subgoal generator and a subgoal-conditioned action prior that CEM refines; FF-JEPA (Masip et al., 2026) learns an action-free latent planner; Hi-LeWM (Caselli et al., 2026) learns macro-actions and a high-level predictor and shows the naive version loses to flat planning because CEM exploits the learned subgoal space. Each of these trains a generative component whose training pairs are same-trajectory windows, so their subgoals cannot lie on a route that no single trajectory took. Our subgoals are dataset frames on a shortest path over the whole dataset, and the cross-episode

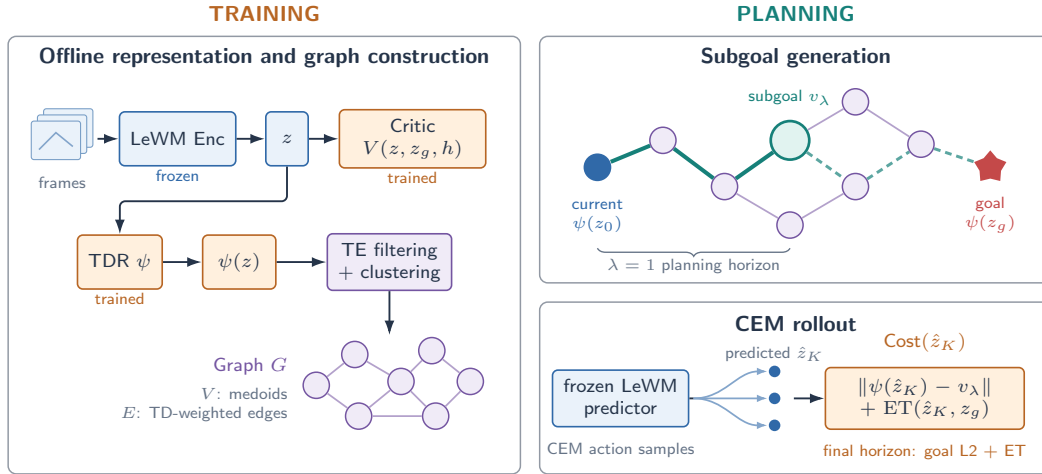


Figure 1: Graph-guided latent MPC. Training (left): We train a Critic: V , TDR ψ and graph G from the dataset frames. Planning (right): Graph identifies a subgoal. CEM scores the candidate rollouts

protocol is where that difference shows. SAGE reports stronger same-episode numbers than ours at long offsets (64.7% at $H = 150$ on Push-T with the same frozen model); we discuss this in section 6.

Graph search over offline data. SPTM (Savinov et al., 2018), SoRB (Eysenbach et al., 2019) and SGM (Emmons et al., 2020) build graphs over replay buffers with a learned distance and plan waypoints for a goal-conditioned policy; GAS (Baek et al., 2025) clusters a temporal distance representation (Park et al., 2024; Wang et al., 2023) into nodes with a temporal-efficiency filter and searches it for subgoals; TTGS (Opryshko et al., 2025) and ALPS (ALPS authors, 2026) do the same at test time for frozen OGBench policies (Park et al., 2025). All of them execute subgoals with a learned policy (or a state-space model with a behaviour prior). We take GAS’s graph construction essentially unchanged and give the subgoal to a pixel world model’s CEM instead.

Value functions inside MPC. POLO (Lowrey et al., 2019), TD-MPC (Hansen et al., 2022; 2024) and Director (Hafner et al., 2022) put a learned value at the end of a short model rollout; goal-conditioned terminal values have been used for multi-task MPC (GC terminal value authors, 2024). Those values are trained with rewards or with the model. Our critic is a reachability classifier trained from hindsight and cross-episode negatives on the offline data, and its contribution is a budget-aware hitting time rather than a value.

3 METHOD

Frozen world model and CEM planner. LeWM (Maes et al., 2026) consists of a ViT encoder E mapping a 224×224 frame to a 192-d CLS latent z and a predictor P that maps three past latents and three action blocks to the next latent, where an action block is five consecutive environment actions (frame-skip 5). Planning is CEM over $K = 5$ blocks (25 environment steps): 300 candidates, 30 iterations, 30 elites, each candidate scored by the terminal cost $c_{L2} = \|\hat{z}_K - z_g\|^2$ between the predicted block- K latent and the encoded goal frame. The optimum is executed open loop for 25 steps and the planner is called again. We keep the encoder, the predictor and this planner fixed throughout and change only the cost function that scores $(z_0, \hat{z}_{1..K}, z_g)$.

Each of these failures are fixed separately. With the planner fixed, terminal L2 fails in two distinct ways beyond one horizon, which we analyze in section 5: a procedural one, already absorbed by the baseline’s 25-step replanning cadence, and a structural one: L2 loses its ranking signal beyond one horizon because the start-goal distance saturates at the random-pair scale. We retain raw L2 within one horizon, via the final switch of section 3.2, and beyond it substitute three quantities

designed to stay informative at range: TDR distance in place of raw position (section 3.1), a graph that follows the shell rather than cutting across it (sections 3.1 and 3.2), and a critic that estimates reachability directly (section 3.3). Figure 1 provides an overview of our approach.

3.1 TEMPORAL DISTANCE REPRESENTATION AND DATASET GRAPH

Let $\mathcal{D} = \{(z, a, z')\}$ denote the offline dataset of consecutive-frame transitions, encoded once by the frozen encoder. We train a temporal distance representation $\psi : \mathbb{R}^{192} \rightarrow \mathbb{R}^{32}$ by expectile TD (HILP/GAS (Park et al., 2024; Baek et al., 2025)): with value $V(z, z_g) = -\|\psi(z) - \psi(z_g)\|$, EMA target V^- , discount $\gamma = 0.99$, TD target $q = -1 + \gamma V^-(z', z_g)$, and, for $(z, a, z') \sim \mathcal{D}$, a goal z_g drawn from z 's own trajectory at a $\text{Geom}(1 - \gamma)$ offset with probability 0.625 and uniformly from $\mathcal{D}$ otherwise,

$$\mathcal{L}_{\text{TDR}} = \mathbb{E}_{(z, a, z') \sim \mathcal{D}, z_g} [\ell_{0.99}(q - V(z, z_g))], \quad \ell_\tau(u) = |\tau - \mathbf{1}_{u < 0}| u^2, \quad (1)$$

(we use HILP/GAS's target-stabilisation trick: computing q over the minimum of two independent networks). At convergence $\|\psi(z) - \psi(z_g)\|$ approximates the discounted minimum step count between z and z_g ; we map distance in TDR space to environment steps using $u_k = \text{median}_t \|\psi(z_t) - \psi(z_{t+k})\|$ over same-episode pairs k steps apart, $k \in \{1, \dots, 50\}$.

The graph follows GAS. Let $F(z_t)$ be the first later frame of t 's trajectory with $\|\psi(F(z_t)) - \psi(z_t)\| \geq H_{\text{TD}}$ (the episode's last frame if none), and define temporal efficiency

$$\text{TE}(z_t) = \cos \angle(\psi(F(z_t)) - \psi(z_t), \psi(z_{t+H_{\text{TD}}}) - \psi(z_t)), \quad (2)$$

the agreement between where a real H_{TD} -step transition moves and the direction ψ calls optimal. Frames with $\text{TE} < \theta_{\text{TE}} = 0.9$ are dropped; the remainder are clustered sequentially in ψ -space – assign to the nearest existing centre if within $H_{\text{TD}}/2$, else open a new one, then reset centres to member means – giving nodes $v \in V_G$ with medoid latent c_v . Nodes within H_{TD} of each other are joined by an edge weighted by their ψ -distance, with $H_{\text{TD}} = u_{12}$. At planning time the goal is attached as a temporary node linked to every node within $\max(H_{\text{TD}}, 1.2 d_{\min})$, and one Dijkstra pass gives the cost-to-go $D_g[v]$ from every node. On Push-T (18,685 expert episodes, 2.34M frames) the graph has 20.5k nodes; Reacher (10k episodes, 2.01M frames) needs 535; Cube (10k episodes, 2.01M frames) needs 16k. Construction is a one-off cost per environment, reported with the rest of the pipeline's compute in section 4.4.

3.2 SUBGOAL ONE HORIZON AHEAD, AND THE FINAL SWITCH

At each planning call with current latent z_0 , we select the start node

$$v^* = \arg \min_{v : \|\psi(z_0) - c_v\| \leq H_{\text{TD}}} D_g[v] + \|\psi(z_0) - c_v\|, \quad (3)$$

and walk the shortest path from v^* toward the goal until the cumulative ψ -distance from z_0 first reaches the lookahead $\lambda = u_{25}$, the calibrated distance of one planning horizon; call the node reached v_λ . The planning criterion is the piecewise cost

$$c(\hat{z}_K) = \begin{cases} \|\hat{z}_K - z_g\|^2 & \text{if } \|\psi(z_0) - \psi(z_g)\| \leq \lambda \quad (\text{final switch}) \\ \|\psi(\hat{z}_K) - c_{v_\lambda}\| =: c_{\text{sub}}(\hat{z}_K) & \text{otherwise.} \end{cases} \quad (4)$$

Two choices here are settled empirically in section 4.3 rather than assumed: the lookahead is exactly one horizon (a shorter one leaves progress on the table, and tracking the intermediate predicted latents $\hat{z}_{1..K-1}$ – the predictor's least reliable outputs – is worse than scoring $\hat{z}_K$ alone), and the final switch matters because a cluster medoid's radius is comparable to the success tolerance while ψ has less resolution near the goal than latent L2, which itself remains discriminative rather than saturated at that range (section D.1: true start-goal distance 17.1 at $H = 25$, clear of the 19.6 random-pair floor, already 19.4 by $H = 50$).

3.3 BUDGET-CLOCKED EXPECTED HITTING TIME

A reachability critic $V(z, z_g, h) \in [0, 1]$, a five-member MLP ensemble over (z, z_g, h) with h on a grid of multiples of 5 up to $h_{\max} = 50$, estimates the probability that the goal is reached from z

within h environment steps. Five label sources supervise it by binary cross-entropy:

- (i) hindsight: $y = \mathbf{1}_{h \geq \tau}$;
- (ii) cross-episode: $y = 0$ if $\|\Delta_{\text{obj}}\| > \delta_{50}$;
- (iii) imagined: predictor rollout under logged actions, labelled as in (i);
- (iv) Bellman: $q = \max_{m \leq M} (\bar{V}(\hat{z}'_m, z_g, h - 5) - \kappa \bar{\sigma}(\hat{z}'_m, z_g, h - 5))_{[0,1]}$;
- (v) monotonicity: $\max(0, V(z, z_g, h) - V(z, z_g, h + 5))$.

Here τ is the first logged step satisfying the success predicate toward a later frame, δ_{50} is the largest displacement the data itself makes in 50 steps (so (ii) labels an object/joint change past that as unreachable), and (v) enters the loss as a hinge penalty. In (iv), $\hat{z}'_m = P(z, a_m)$ rolls the frozen predictor forward under M candidate action blocks a_m sampled around the logged action, and $\bar{V}, \bar{\sigma}$ are the mean and standard deviation of an EMA target ensemble, clipped to $[0, 1]$ before the max so that a single head’s optimism on an off-manifold state cannot dominate. Labels for (i) and the negative-sampling gate in (ii) are read from the dataset’s state column, not from pixels: although the critic’s inputs at inference are latents only, its supervision is state-dependent rather than pixel-only. Section 6 discusses what this costs.

Rather than the usual $-\log V$ penalty, we score an endpoint by how long the goal is expected to take from it, capped at the budget that will remain once the plan being scored has been executed:

$$\text{ET}(\hat{z}_K, z_g) = 5 \sum_{h \in \{0, 5, \dots, h_{\text{rem}}\}} (1 - V(\hat{z}_K, z_g, h)), \quad h_{\text{rem}} = \min(h_{\text{max}}, B - t - 5K), \quad (5)$$

where B is the episode budget and t the current step, supplied by a *plan clock* that advances by one horizon with every CEM call. Without it, the critic would be queried at a fixed h regardless of steps already spent, understating reachability whenever the true remaining budget is smaller – most severely near the end of an episode; the plan clock instead asks, at every call, “reachable in the steps that are actually left.” Feasible endpoints are then ordered by how soon they arrive; infeasible ones sit at the cap. The final criterion standardises both terms by median and interquartile range over the 300 CEM candidates of each environment (denoted $\tilde{\cdot}$) and adds them,

$$c = \widetilde{c_{\text{sub}}} + \beta \widetilde{\text{ET}}, \quad \beta = 1, \quad (6)$$

with c_{L2} in place of c_{sub} in the final phase. Algorithm 1 (section A) summarises one planning call.

What is frozen The TDR (50k steps, batch 1024) and the critic (60k steps) are the only trained components; parameter counts and training cost are reported with the rest of the pipeline’s compute in section 4.4. The encoder, predictor, CEM hyperparameters, and graph-construction rules are LeWM’s and GAS’s, unchanged. Two scalars are environment specific, $H_{\text{TD}} = u_{12}$ and $\lambda = u_{25}$, and both are read off the TDR calibration table rather than tuned; $\beta = 1$, $\theta_{\text{TE}} = 0.9$ and $h_{\text{max}} = 50$ are shared by every environment and protocol.

4 EXPERIMENTS

4.1 SETUP

Environments and data. Push-T (Maes et al., 2026): 18,685 expert episodes; success when agent and block position errors are below 20 px and block angle error below $\pi/9$. Reacher (dm.control): 10,000 episodes of 201 frames; success when both joints are within 0.05 rad of the goal configuration (raw difference, no angle wrap; the dataset’s shoulder angle is unwrapped over ± 8 rad). OGBench Cube (Park et al., 2025): 10,000 episodes of 201 frames; success when the cube’s position is within 0.04 m of its target (CubeEnv.compute_successes, position only, no orientation or effector-pose requirement). Frozen LeWM checkpoints are the public ones for each environment.

Protocols. For a same-episode offset K , the goal is the frame K steps after the start on the same trajectory and the budget is $2K$ steps. For the cross-episode protocol the start and goal are uniformly random frames of two different episodes with no distance or reachability filter and a budget of one full episode. We use a fixed set of 50 tasks per protocol for every method and every seed. Each seed

Table 1: Success rate (%) on the fixed, held-out tasks, mean $\pm$ sd over five seeds. Within each environment, **bold** is the best mean and underlined means having a $\pm$ sd interval that overlaps the best’s.

Env	Method	same-ep 25	same-ep 50	same-ep 100	cross-ep
Push-T	Random	4.0 $\pm$ 1.3	0.4 $\pm$ 0.8	0.0 $\pm$ 0.0	0.4 $\pm$ 0.8
	L2 (LeWM)	91.2 $\pm$ 1.0	40.8 $\pm$ 3.7	13.2 $\pm$ 3.2	14.4 $\pm$ 3.9
	Ours	<u>88.4</u> $\pm$ 2.3	70.0 $\pm$ 2.2	46.4 $\pm$ 3.2	42.4 $\pm$ 3.2
Reacher	Random	8.0 $\pm$ 3.8	8.4 $\pm$ 0.8	9.2 $\pm$ 3.2	2.8 $\pm$ 1.0
	L2 (LeWM)	77.2 $\pm$ 4.8	90.8 $\pm$ 1.0	82.8 $\pm$ 1.0	29.2 $\pm$ 2.4
	Ours	<u>76.0</u> $\pm$ 4.6	98.0 $\pm$ 1.3	97.6 $\pm$ 0.8	65.6 $\pm$ 2.0
Cube	Random	21.2 $\pm$ 3.0	12.8 $\pm$ 2.0	4.8 $\pm$ 4.1	3.6 $\pm$ 2.3
	L2 (LeWM)	<u>56.0</u> $\pm$ 3.8	34.4 $\pm$ 3.2	35.6 $\pm$ 1.5	16.4 $\pm$ 1.5
	Ours	62.0 $\pm$ 4.7	49.6 $\pm$ 4.6	50.4 $\pm$ 3.2	27.6 $\pm$ 6.1

Table 2: Push-T ablation (success %, same tasks and seeds as table 1). Rows (a)–(c) build the graph subgoal without a critic; (d) adds the critic without a graph; (e)–(g) add a critic to the graph subgoal; (h) scores the subgoal with LeWM’s L2 instead of TDR distance. Full ladder in table 5. **bold**: best mean per column; underlined: $\pm$ sd interval overlaps the best’s.

	configuration	same-ep 25	same-ep 50	same-ep 100	cross-ep
	L2 (LeWM)	91.2 $\pm$ 1.0	40.8 $\pm$ 3.7	13.2 $\pm$ 3.2	14.4 $\pm$ 3.9
(a)	TDR terminal cost (no graph)	83.2 $\pm$ 4.1	48.4 $\pm$ 1.5	22.4 $\pm$ 5.3	14.0 $\pm$ 1.3
(b)	graph subgoal	80.8 $\pm$ 2.7	51.6 $\pm$ 2.3	22.4 $\pm$ 5.0	15.2 $\pm$ 3.0
(c)	(b) + final switch	<u>88.0</u> $\pm$ 2.5	62.4 $\pm$ 2.7	30.8 $\pm$ 6.5	17.2 $\pm$ 2.7
(d)	L2 + ET critic (no graph)	<u>89.2</u> $\pm$ 3.0	58.4 $\pm$ 5.1	<u>38.4</u> $\pm$ 7.1	31.6 $\pm$ 2.3
(e)	(b) + ET critic (no switch)	86.0 $\pm$ 3.4	<u>64.8</u> $\pm$ 3.0	<u>43.6</u> $\pm$ 5.0	35.2 $\pm$ 2.0
(f)	(c) + $-\log V$ critic	77.6 $\pm$ 1.5	<u>69.2</u> $\pm$ 2.4	47.2 $\pm$ 5.9	<u>34.4</u> $\pm$ 7.3
(g)	(c) + ET critic (ours)	88.4 $\pm$ 2.3	70.0 $\pm$ 2.2	<u>46.4</u> $\pm$ 3.2	42.4 $\pm$ 3.2
(h)	(g), subgoal scored by L2	86.0 $\pm$ 2.2	<u>63.6</u> $\pm$ 4.8	<u>45.2</u> $\pm$ 3.5	<u>36.4</u> $\pm$ 2.9

retrains the learned assets (TDR, graph, critic) and reseeds CEM sampling, so reported standard deviations include both sources of variation; L2 has no learned assets, and its seeds vary CEM sampling only.

Baselines. L2 is LeWM’s planner. TDR scores endpoints by $\|\psi(\hat{z}_K) - \psi(z_g)\|$ with no graph; CTG uses the graph cost-to-go $\min_v \|\psi(\hat{z}_K) - c_v\| + D_g[v]$ as a terminal value with no subgoal commitment. Random takes a uniform action from the env’s own action space at every step (no model, no planning): the floor L2 and Ours must clear (table 1).

4.2 MAIN RESULT

On Push-T (table 1) the method is 2.8 points under L2 at same-ep 25 (88.4 vs 91.2), gains 29 points at same-ep 50 (70.0 vs 40.8), and is the strongest configuration in our study at same-ep 100 and on cross-episode goals (46.4 and 42.4 against 13.2 and 14.4). On Reacher, L2’s success rate holds up well for same-episode goals across horizons (77.2/90.8/82.8) but is poor cross-episode (29.2); the method matches this same-episode performance (76.0/98.0/97.6) and reaches 65.6% cross-episode, a +36-point gain. At same-ep 25 on Reacher the method is 1.2 points under L2. On Cube the method wins at every protocol, including same-ep 25 (62.0 vs 56.0) where Push-T and Reacher do not: +6.0 at same-ep 25, +15.2 at same-ep 50, +14.8 at same-ep 100, and +11.2 on cross-episode goals.

4.3 ABLATION LADDER

Table 2 adds one component at a time to Push-T, on the same task pool and seeds as table 1. The full ladder, including the β and H_{TD} sweeps, is in section B.

Table 3: Push-T success (%) in varying conditions: **(a)** Under loosened success criteria: Our method recovers more failures. **(b)** Cumulative success through each 50-step budget window; in brackets, the hazard rate: the share of tasks still unsolved before entering the window that are solved within it.

(a) Success threshold			(b) Success by budget window				
threshold	Ours	L2	window (steps)	same-ep 100		cross-ep	
				Ours	L2	Ours	L2
<i>same-ep 100</i>			0–50	8.8 (8.8)	8.8 (8.8)	14.8 (14.8)	6.0 (6.0)
20 px/20°	46.4	13.2	50–100	23.2 (15.8)	12.0 (3.5)	30.4 (18.3)	12.0 (6.4)
25 px/25°	52.8	14.8	100–150	36.4 (17.2)	12.4 (0.5)	36.4 (8.6)	13.2 (1.4)
30 px/30°	58.0	16.0	150–200	46.4 (15.7)	13.2 (0.9)	38.8 (3.8)	14.0 (0.9)
<i>cross-ep</i>			200–250	–	–	42.4 (5.9)	14.4 (0.5)
20 px/20°	42.4	14.4					
25 px/25°	51.2	19.2					
30 px/30°	57.6	22.8					

Direction: the graph subgoal. A graph subgoal (b) lifts offset 50 and 100 over L2 (40.8 $\rightarrow$ 51.6%, 13.2 $\rightarrow$ 22.4%). The TDR metric alone (a) gets part of this, so the graph’s route adds to the metric, not just replaces L2’s geometry. At offset 50, L2’s closest approach is 32 px but it drifts to a final error of 76 px; with the subgoal, closest approach and final error are both 19 px. The route matters more than how the subgoal is scored. In (h) the planner follows the same graph and subgoal node, but ranks candidates by LeWM’s L2 distance to the node’s medoid latent instead of TDR distance. It keeps most of the full method’s gain at offset 100 (45.2 vs. 46.4%) and loses about six points at offset 50 and cross-episode (63.6 vs. 70.0%, 36.4 vs. 42.4%).

Resolution: the final switch. Handing the last horizon back to c_{L2} (c) closes most of the same-ep 25 gap (80.8 $\rightarrow$ 88.0%) and lifts offset 50 and 100 further. On its own it leaves cross-episode unchanged (17.2%): resolution near the goal does not tell the planner which candidates can reach it. With the critic in place the switch still matters. Removing it (e vs. g) costs 2–7 points on every protocol, the most at cross-episode (35.2 vs. 42.4%). This is consistent with the medoid-radius floor: without the switch, the planner stops short of goals the critic has already judged reachable.

Reachability: the critic. The critic supplies most of the cross-episode gain. On top of L2 alone (d) it doubles cross-episode success (14.4 $\rightarrow$ 31.6%); on top of the graph (g) it reaches 42.4%. The graph therefore adds 8–12 points over the critic alone at offset 50, 100 and cross-episode, and the critic adds 8–25 points over the graph alone. Without the critic the graph finds a route but the block drifts once the approach begins (median final error 125 px vs. 26 px with the critic).

ET versus $-\log V$. The two critic costs tie at long range, but $-\log V$ (f) loses 11 points at offset 25. There the critic saturates (67% of the CEM population is feasible, versus 39–41% at offset 100 and cross-episode), and per-population standardisation turns that saturation into a full vote. The capped hitting time keeps ranking feasible candidates by how soon they arrive. With ET, a single $\beta = 1$ is best or within one standard deviation of the best on every protocol (table 5).

4.4 MECHANISTIC ANALYSIS

We give three explanations for the results in table 1. Table 3 compares Push-T success, at same-ep 100 and cross-episode, in varying conditions.

L2 ranks well only at short range; the graph degrades more gracefully. Scoring CEM’s own first-iteration elites against the privileged recovery-time oracle (section E), L2’s cost is the better ranker at same-ep 25 (Spearman 0.396 vs. 0.327 for the graph cost) but collapses beyond it, to 0.086 at same-ep 50 and -0.064 at same-ep 100 – statistically indistinguishable from chance. The graph cost degrades far more gracefully over the same range (0.327 $\rightarrow$ 0.203 $\rightarrow$ 0.191). This is the justification for the switch to L2 within the final horizon (equation (4)): L2 is the better signal at short range, and the graph is preferred at longer horizons where L2’s ranking fails.

Our failures are closer misses than L2’s. Loosening Push-T’s success threshold from its default 20 px/20° to 25 px/25° and 30 px/30° (table 3a) recovers far more of Our failures than L2’s: at

Table 4: Push-T compute cost on one GTX 1080 Ti. Preparation is a one-off ~ 3.2 h, mostly critic training; planning with the graph and critic takes $1.6\times$ L2’s time per decision.

<i>One-off preparation (GPU-minutes, per seed)</i>	
Encode (shared across seeds)	49.6
TDR training (50k steps)	16.6
Graph construction	1.6
Critic training (60k steps)	123.0
Total (incl. shared encode)	190.8
<i>Per-decision planning time (s)</i>	
L2	1.14
Ours (graph subgoal + critic)	1.80
Ratio vs. L2	$1.58\times$

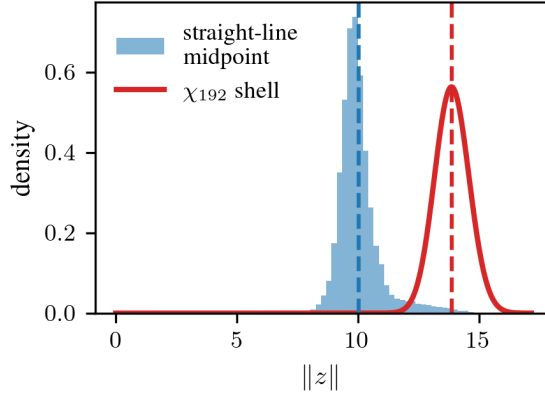


Figure 2: Straight-line midpoints leave the data distribution. Push-T, same-episode pairs $(z_{\text{start}}, z_{\text{goal}})$, 100 steps apart: radius of midpoints $\|\frac{1}{2}(z_{\text{start}} + z_{\text{goal}})\|$ (blue) vs. the χ_{192} shell that encoded frames lie in (red).

same-ep 100, Ours rises $46.4 \rightarrow 52.8 \rightarrow 58.0\%$ while L2 moves only $13.2 \rightarrow 14.8 \rightarrow 16.0\%$; at cross-episode, Ours rises $42.4 \rightarrow 51.2 \rightarrow 57.6\%$ against L2’s $14.4 \rightarrow 19.2 \rightarrow 22.8\%$. The Ours-over-L2 gap widens rather than narrows under a looser criterion, so the advantage in table 1 is not an artifact of the strict threshold disproportionately penalizing Our near-misses – when Ours fails, it still remains closer to the goal than L2 does.

L2 stalls after its first replan; Ours keeps converging. Binning success by cumulative environment-step budget (table 3b) shows L2’s hazard rate – the fraction of still-unsolved tasks solved within each successive 50-step window – collapses to under 3.5% after the first window at both same-ep 100 and cross-episode: nearly all of L2’s successes come from its first two plans, and additional budget shows marginal improvement (cross-episode success rises only from 12.0% at 100 steps to 14.4% at 250). Our hazard rate instead stays between 8.6% and 18.3% through same-ep 100’s entire budget and cross-episode’s first three windows, so a substantial share of the tasks still unsolved keep getting solved by later replans. This explains how Our advantage in table 1 accumulates: it is able to maintain sustained progress at later steps, while L2 is incapable of doing so.

Compute. Table 4 reports preparation and planning cost; parameter counts are in table 9. Both learned additions are small next to the frozen encoder and predictor (18.03M): the TDR net used at planning time is 0.64M and the five-member critic 4.13M. The Dijkstra pass runs once per goal, not per candidate. Together, they add 58% to the planning time, with respect to L2.

5 WHY TERMINAL L2 FAILS BEYOND ONE HORIZON

The L2 rows of table 1 fall from 91.2% at offset 25 to 40.8% at 50, 13.2% at 100 and 14.4% cross-episode. We separate two causes.

Long open-loop rollouts compound predictor error. Widening LeWM’s default 2×25 -step rollouts naively to $2 \times H$ -step rollouts for longer offsets lowers L2’s success from 91.2% at offset 25 to 34.4% at offset 50 and 5.6% at offset 100 (section D.1). The predictor’s relative rollout error grows from 14.1% of the start–goal distance at $H = 25$ to 39.8% at $H = 100$. This failure is procedural: replanning every 25 steps minimizes accumulated predictor error, and every L2 result in this paper uses that cadence.

The terminal distance saturates. Replanning does not rescue the cost itself. In LeWM’s Push-T latent, two unrelated frames sit 19.6 apart on average. True same-episode start–goal distance is 17.1 at $H = 25$ but already 19.4 at $H = 50$ and 19.6 at $H = 100$ (section D.1): beyond one horizon, the goal is as far in L2 as a random frame, so terminal distance carries little information about which candidate is closer in time. This is consistent with the degradation of correlation between the L2 scores of candidates against the oracle observed in section 4.4 .

L2 measures distance through a region the data does not occupy. The saturation has a geometric cause. section D.2 shows that, in high dimensions, an isotropic Gaussian puts almost all its mass on a thin shell of radius $\sigma\sqrt{d}$ ($= 13.88$) around the origin. figure 4 verifies this for LeWM’s Push-T latents. Two points on a shell are joined by a chord through its empty interior, while real trajectories stay on the shell. figure 2 shows that for same-episode pairs 100 steps apart, the straight-line midpoint $\frac{1}{2}(z_{\text{start}} + z_{\text{goal}})$ lies at radius 10.03, and 96.4% of them fall below the shell’s 1st percentile. Terminal L2 optimizes along this chord: it measures distance through states the data never visits, not the distance the dynamics must travel along the shell. The TDR and the graph (section 3.1) are built to follow the data instead.

6 LIMITATIONS

All results use one frozen model family; whether the same graph and critic help DINO-WM or PLDM is untested. SAGE (Cheng et al., 2026) reports higher same-episode success than ours at long offsets with the same frozen model by training a subgoal generator on 400k expert windows; we do not claim the same-episode state of the art, and we expect a generator trained on single-trajectory windows to be unable to stitch, and thus not generalize adequately to cross-episode. Reacher’s short-range loss (six points at offset 25) is inside CEM noise over ten seeds but consistent in sign. All data are expert demonstrations; the suboptimal-data regime where graph stitching is usually studied is not covered.

7 CONCLUSION

A frozen pixel world model’s planner fails past one horizon because its terminal cost cannot tell which candidates can still reach the goal. Giving it a data-supported target one horizon along a shortest path over the dataset, a budget-aware hitting-time estimate of what remains, and its own objective for the last horizon repairs mid-range, long-range and cross-episode goal reaching with nothing in the model or the planner retrained. The three pieces fix three different failures, and the per-pair diagnostics say why the last one is necessary: the graph knows a route for every cross-episode pair, and the planner cannot execute it without a reachability term.

AI USE STATEMENT

In this work, we used generative AI tools (an LLM-based coding assistant) to write and refactor implementation and analysis code, to help with literature search, and to edit text. We have not used generative AI tools to generate synthetic data, to develop the conceptual framework, or to formulate mathematical claims. We have reviewed all AI-assisted work: LLM-written code was read and tested by the authors, every reported number was produced by runs the authors launched and checked, and citations found with AI assistance were verified against the original papers. We take responsibility for the final content of this work, including text, claims or artifacts produced with the aid of generative AI. **[TODO: Confirm the disclosure list with all authors.]**

ETHICS STATEMENT

This work uses simulated benchmarks with public datasets and checkpoints and involves no human subjects or sensitive data.

REPRODUCIBILITY STATEMENT

[TODO: Point to the anonymised code, the fixed task-pool files, the seeds, the TDR / graph / critic checkpoints, and the exact CEM configuration. Mention the MuJoCo render-version check (section C) and the action-normalisation convention (predictor inputs are z-scored with the full dataset’s action statistics).]

REFERENCES

- ALPS authors. Laplacian representations for decision-time planning. In *International Conference on Machine Learning (ICML)*, 2026. verify authors; code at github.com/machado-research/ALPS.
- Mahmoud Assran et al. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025. verify.
- Seungho Baek, Taegeon Park, Jongchan Park, Seungjun Oh, and Yusung Kim. Graph-assisted stitching for offline hierarchical reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2025. [arXiv:2506.07744](https://arxiv.org/abs/2506.07744).
- Jiaxin Bai and Jiaxuan Xiong. Temporal-distance JEPA: Plan-aware representation learning for latent world model predictive control. *arXiv preprint arXiv:2607.25337*, 2026.
- Caselli, Massafra, Punzo, Lo Sardo, Pantelidis, and Bhethanabhotla. Mind the gap: Promises and pitfalls of hierarchical planning in LeWorldModel. *arXiv preprint arXiv:2607.12547*, 2026. verify author first names.
- Letian Cheng, Qi Zhang, and Yisen Wang. SAGE: Subgoal-conditioned action generation for latent world model planning. *arXiv preprint arXiv:2607.17973*, 2026.
- Matthieu Destrade, Oumayma Bounou, Quentin Le Lidec, Jean Ponce, and Yann LeCun. Value-guided action planning with JEPA world models. *arXiv preprint arXiv:2601.00844*, 2026. World Modeling Workshop 2026.
- Scott Emmons, Ajay Jain, Michael Laskin, Thanard Kurutach, Pieter Abbeel, and Deepak Pathak. Sparse graphical memory for robust planning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- Benjamin Eysenbach, Ruslan Salakhutdinov, and Sergey Levine. Search on the replay buffer: Bridging planning and reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- GC terminal value authors. Goal-conditioned terminal value estimation for real-time and multi-task model predictive control. *arXiv preprint*, 2024. verify: locate the arXiv id.
- Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *International Conference on Machine Learning (ICML)*, 2019.
- Danijar Hafner, Kuang-Huei Lee, Ian Fischer, and Pieter Abbeel. Deep hierarchical planning from pixels. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.
- Nicklas Hansen, Xiaolong Wang, and Hao Su. Temporal difference learning for model predictive control. In *International Conference on Machine Learning (ICML)*, 2022.
- Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations (ICLR)*, 2024.
- Jiaming Hu, Yan Zheng, and Tian Wang. SCALE: State-calibrated latent embeddings for JEPA planning in the right geometry. *arXiv preprint arXiv:2608.16287*, 2026.

- Liangyu Li, Shengzhi Wang, Libin Qiu, Mingliang Xiong, and Qingwen Liu. Beyond Euclidean proximity: Repairing latent world models with horizon-matched trajectory reachability metrics. *arXiv preprint arXiv:2605.22164*, 2026a.
- Wenyuan Li, Guang Li, Keisuke Maeda, Takahiro Ogawa, and Miki Haseyama. Predictive but not plannable: RC-aux for latent world models. *arXiv preprint arXiv:2605.07278*, 2026b.
- Kendall Lowrey, Aravind Rajeswaran, Sham Kakade, Emanuel Todorov, and Igor Mordatch. Plan online, learn offline: Efficient learning and exploration via model-based control. In *International Conference on Learning Representations (ICLR)*, 2019.
- Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. LeWorld-Model: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026.
- Sergi Masip, Jonathan Swinnen, Yutong Hu, Renaud Detry, and Tinne Tuytelaars. FF-JEPA: Long-horizon planning in world models with latent planners. *arXiv preprint arXiv:2606.09311*, 2026.
- Evgenii Opryshko, Junwei Quan, Claas Voelcker, Yilun Du, and Igor Gilitschenski. Test-time graph search for goal-conditioned reinforcement learning. *arXiv preprint arXiv:2510.07257*, 2025.
- Seohong Park, Tobias Kreiman, and Sergey Levine. Foundation policies with Hilbert representations. In *International Conference on Machine Learning (ICML)*, 2024.
- Seohong Park, Kevin Frans, Benjamin Eysenbach, and Sergey Levine. OGBench: Benchmarking offline goal-conditioned RL. In *International Conference on Learning Representations (ICLR)*, 2025.
- Nikolay Savinov, Alexey Dosovitskiy, and Vladlen Koltun. Semi-parametric topological memory for navigation. In *International Conference on Learning Representations (ICLR)*, 2018.
- Vlad Sobal, Wancong Zhang, Kynghyun Cho, Randall Balestriero, Tim G. J. Rodriguez, and Yann LeCun. Learning from reward-free offline data: A case for planning with latent dynamics models. *arXiv preprint arXiv:2502.14819*, 2025. verify author list.
- stable-worldmodel authors. stable-worldmodel: A platform for reproducible world modeling research and evaluation. *arXiv preprint arXiv:2605.21800*, 2026. verify authors.
- Temporal straightening authors. Temporal straightening for latent planning. *arXiv preprint arXiv:2603.12231*, 2026. verify authors.
- Basile Terver, Tsung-Yen Yang, Jean Ponce, Adrien Bardes, and Yann LeCun. What drives success in physical planning with joint-embedding predictive world models? *arXiv preprint arXiv:2512.24497*, 2025.
- Jiawei Wang, Ke Rui, Yushen Zuo, Yichun Feng, and Minglei Li. Decision-metric alignment in latent world models: Diagnostics and action-conditioned objectives for MPC planning. *arXiv preprint arXiv:2608.18746*, 2026.
- Tongzhou Wang, Antonio Torralba, Phillip Isola, and Amy Zhang. Optimal goal-reaching reinforcement learning via quasimetric learning. In *International Conference on Machine Learning (ICML)*, 2023.
- Wancong Zhang, Basile Terver, Artem Zhohus, Soham Chitnis, Harsh Sutaria, Mido Assran, Randall Balestriero, Amir Bar, Adrien Bardes, Yann LeCun, and Nicolas Ballas. Hierarchical planning with latent world models. *arXiv preprint arXiv:2604.03208*, 2026.
- Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning. *arXiv preprint arXiv:2411.04983*, 2024.

A PLANNING ALGORITHM

Algorithm 1 One planning call of graph-assisted latent MPC

Require: frozen E, P ; CEM; TDR ψ ; graph (V_G, c_v, D_g) ; critic V ; budget B ; step t

- 1: $z_0 \leftarrow E(o_t)$; $h_{\text{rem}} \leftarrow \min(h_{\text{max}}, B - t - 5K)$
- 2: **if** $\|\psi(z_0) - \psi(z_g)\| \leq \lambda$ **then**
- 3: target $\leftarrow$ goal; base cost $\leftarrow c_{L2}$
- 4: **else**
- 5: $v^* \leftarrow \arg \min_{v: \|\psi(z_0) - c_v\| \leq H_{\text{TD}}} D_g[v] + \|\psi(z_0) - c_v\|$
- 6: walk the shortest path from v^* until cumulative TDR distance $\geq \lambda$; target $\leftarrow v_\lambda$; base cost $\leftarrow c_{\text{sub}}$
- 7: **end if**
- 8: **for** each CEM iteration **do**
- 9: sample 300 action-block sequences; $\hat{z}_{1..K} \leftarrow P(\cdot)$
- 10: $c \leftarrow \widetilde{\text{base}}(\hat{z}_K) + \widetilde{\text{ET}}(\hat{z}_K, z_g)$ with ET from equation (5)
- 11: refit the CEM Gaussian on the 30 lowest- c candidates
- 12: **end for**
- 13: execute the mean sequence for $5K$ steps; $t \leftarrow t + 5K$

B ABLATION LADDER

Table 5: Full Push-T ablation ladder (50 tasks, five seeds, mean $\pm$ sd). Both indented rows build on the graph subgoal row above them; A is the graph subgoal with the final switch.

configuration	same-ep 25	same-ep 50	same-ep 100	cross-ep
L2 (LeWM)	91.2 $\pm$ 1.0	40.8 $\pm$ 3.7	13.2 $\pm$ 3.2	14.4 $\pm$ 3.9
TDR terminal cost (no graph)	83.2 $\pm$ 4.1	48.4 $\pm$ 1.5	22.4 $\pm$ 5.3	14.0 $\pm$ 1.3
graph cost-to-go (no subgoal)	83.2 $\pm$ 4.1	47.2 $\pm$ 1.6	21.6 $\pm$ 4.6	14.0 $\pm$ 1.3
graph subgoal, lookahead u_{25}	80.8 $\pm$ 2.7	51.6 $\pm$ 2.3	22.4 $\pm$ 5.0	15.2 $\pm$ 3.0
+ ET critic, $\beta = 1$ (no switch)	86.0 $\pm$ 3.4	64.8 $\pm$ 3.0	43.6 $\pm$ 5.0	35.2 $\pm$ 2.0
+ final switch at λ , L2 in final phase (A)	88.0 $\pm$ 2.5	62.4 $\pm$ 2.7	30.8 $\pm$ 6.5	17.2 $\pm$ 2.7
A + critic $-\log V$, $\beta = 1$ (old)	77.6 $\pm$ 1.5	69.2 $\pm$ 2.4	47.2 $\pm$ 5.9	34.4 $\pm$ 7.3
A + ET critic, $\beta = 0.5$	88.4 $\pm$ 2.0	69.2 $\pm$ 2.7	43.2 $\pm$ 3.7	28.0 $\pm$ 3.8
A + ET critic, $\beta = 1$ (ours)	88.4 $\pm$ 2.3	70.0 $\pm$ 2.2	46.4 $\pm$ 3.2	42.4 $\pm$ 3.2
A + ET critic, $\beta = 2$	87.6 $\pm$ 3.2	72.4 $\pm$ 4.1	42.8 $\pm$ 4.1	35.2 $\pm$ 3.5
A + ET critic, $\beta = 4$	86.8 $\pm$ 2.0	65.6 $\pm$ 2.3	48.4 $\pm$ 5.0	38.4 $\pm$ 3.4
L2 + ET critic, $\beta = 1$ (no graph)	89.2 $\pm$ 3.0	58.4 $\pm$ 5.1	38.4 $\pm$ 7.1	31.6 $\pm$ 2.3
graph subgoal (non-TDR) + final switch + ET critic, $\beta = 1$	86.0 $\pm$ 2.2	63.6 $\pm$ 4.8	45.2 $\pm$ 3.5	36.4 $\pm$ 2.9
A + ET critic, $\beta = 1$, $H_{\text{TD}} = 4$	82.8 $\pm$ 2.0	64.0 $\pm$ 1.3	48.4 $\pm$ 6.1	35.2 $\pm$ 2.7
A + ET critic, $\beta = 1$, $H_{\text{TD}} = 12$	89.6 $\pm$ 2.0	68.0 $\pm$ 3.8	48.0 $\pm$ 4.0	32.4 $\pm$ 6.1
A + ET critic, $\beta = 1$, $H_{\text{TD}} = 16$	91.2 $\pm$ 2.4	64.4 $\pm$ 6.6	47.6 $\pm$ 3.4	36.8 $\pm$ 4.5

C LIVE RENDERING MUST MATCH THE DATASET

The frozen encoder is sensitive to renderer versions. Reacher frames rendered with MuJoCo 3.12 lack the floor checker texture of the dataset’s MuJoCo 3.5 frames; the pixel difference is 18/255 on average and moves the latent by a median of 15 units, about one 25-step gap, so the terminal cost never reaches zero even with the arm at the goal. This alone took LeWM’s own planner from 87% to 65% at offset 25. Our launcher refuses to evaluate unless three dataset frames re-render to within 4/255 of the stored pixels; Push-T (pygame) and Cube pass with 0.5/255 and $\leq 0.2/255$.

D WHERE THE LATENT-L2 OBJECTIVE FAILS

This section traces one failure mode end to end: empirically, in how far apart start/goal pairs actually sit in latent space as the horizon grows (section D.1); and mechanistically, in why a straight line between two such points leaves the region the encoder actually populates (section D.2). Section E follows with a separate, operational check of the same failure inside CEM’s own candidate ranking.

D.1 PREDICTOR ROLLOUT ERROR BY HORIZON

Rolling the frozen predictor forward under real logged actions and comparing to the true encoded endpoint, Push-T’s latent rollout error grows with horizon but stays well below the scale of two unrelated frames (mean 19.6) even at $H = 100$ (table 6 and figure 3): the predictor degrades gracefully rather than collapsing to an uninformative guess. This matches theory: section D.2’s isotropic-Gaussian fit to the latent cloud ($d = 192$, empirical $\|z - \mu\|$ mean 13.88) predicts a random-pair distance of $\sqrt{2} \times 13.88 \approx 19.63$, since $z_1 - z_2 \sim \mathcal{N}(0, 2\sigma^2 I_d)$ for two independent draws – essentially exact agreement with the measured 19.6.

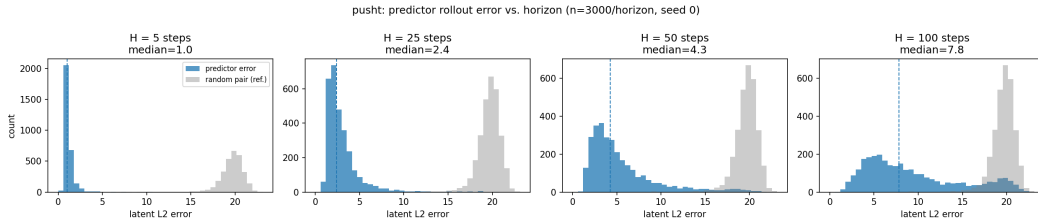


Figure 3: Push-T predictor latent rollout error (blue) vs. horizon, against the random-pair reference distribution (gray).

Task difficulty: how far apart are the actual start/goal pairs? The random-pair reference above (mean 19.6 on the 3,000-sample draw used for figure 3) is the scale at which two frames carry no information about each other. Sampling same-episode (start, start+ H) pairs the same way figure 3 samples its rollout starts, but reading off the true encoded distance $\|z_{\text{start}} - z_{\text{start}+H}\|$ instead of rolling the predictor forward (table 6), shows $H = 5$ pairs sit far below the random-pair scale (median 5.3, std 2.8) while $H = 25$ is already most of the way there (17.1) and $H = 50/H = 100$ are close to indistinguishable from it (19.4, 19.6) – consistent with section 5’s Spearman collapse past $H=50$: an arbitrarily sampled goal at that range is close to latent-distance-indistinguishable from a random unrelated frame, exactly the regime where a cost that only reads terminal z -space L2 has nothing left to discriminate on. Relative to that task scale, the predictor’s own rollout error is already 18.9% of the gap at $H = 5$ and grows to 39.8% by $H = 100$ – a sizeable and growing fraction of the very distance CEM is trying to cross, consistent with table 1’s OUR-vs-L2 gap widening at longer horizons. The task200u pool used for evaluation agrees closely with this same-episode sampling wherever the two overlap (same-ep 25/50/100 medians 17.2/19.6/19.5, std 4.1/3.7/2.1), so the two measurements of task difficulty are not an artifact of either sampling scheme.

Table 6: Push-T: predictor latent rollout error vs. the true same-episode start→goal distance at the same horizon (both $n = 3,000/\text{horizon}$, identically sampled), median $\pm$ std in latent L2. Random-pair reference: mean 19.6 (theory: $\sqrt{2} \times 13.88 \approx 19.63$, section D.2). Relative error is the predictor-error median divided by the start→goal-distance median.

	$H = 5$	25	50	100
predictor error	1.00 ± 0.56	2.41 ± 2.09	4.28 ± 3.82	7.80 ± 5.35
start→goal distance	5.29 ± 2.76	17.06 ± 4.19	19.45 ± 3.29	19.60 ± 2.43
relative error (%)	18.9	14.1	22.0	39.8

Scripts: `scripts/task_start_goal_distance.py` (*task200u pool, per protocol*) and `scripts/direct_horizon_distance.py` (*same-episode sampling at a fixed horizon, reusing predictor_error_by_horizon.py's own sampler*).

This still leaves the planning-side question: does the L2 baseline do better with frequent short replans or fewer wide ones? Same-ep 25's budget (2×25) already gives the default cadence a $2 \times H$ structure at $H = 25$; table 7 extends it to larger H by widening the chunk to match the offset.

Table 7: Push-T L2 baseline: default 25-step-chunk cadence vs. chunks widened to the offset ($2 \times H$, or $3 \times H$ for cross-ep). Success (%), mean $\pm$ sd over 5 CEM seeds.

protocol	chunking	mean $\pm$ sd
same-ep 25	default = 2×25	91.2 ± 1.1
same-ep 50	default (4×25) $2 \times H$ (2×50)	40.8 ± 4.1 34.4 ± 3.0
same-ep 100	default (8×25) $2 \times H$ (2×100)	13.2 ± 3.6 5.6 ± 2.5
cross-ep	default (10×25) $3 \times H$ (3×100)	14.4 ± 4.3 4.8 ± 2.2

Same-ep 25's 91.2% sits where the two cadences coincide. Past $H = 25$, widening the chunk toward $2H$ costs the L2 baseline 6–10 points, growing with how much of the budget one chunk covers – consistent with predictor error compounding across an open-loop commitment, and confirming the default 25-step cadence used elsewhere in this paper is the more favorable setting for L2, not an artifact inflating the OUR-vs-L2 gaps in table 1.

D.2 WHY A STRAIGHT LATENT LINE GOES OUT OF DISTRIBUTION

Section D.1 shows the predictor itself degrades gracefully with horizon; this section asks why the terminal L2 objective built on top of it still fails past $H \approx 50$ (elite-ranking Spearman $0.396 \rightarrow 0.086$, section 5). In a high-dimensional isotropic Gaussian, essentially all probability mass concentrates on a thin shell at radius $\sigma\sqrt{d}$ rather than near the mean, because the volume element $r^{d-1} dr$ grows faster than the Gaussian density falls off until $r \approx \sigma\sqrt{d}$.¹ If LeWM's Push-T latent cloud approximates such a shell, a straight line drawn between two points *on* the shell passes through the ball's interior, which the shell distribution assigns almost no mass – exactly the regime a z -space-L2 CEM plan implicitly cuts through when it scores a candidate purely by terminal distance to a goal many steps away.

The latent cloud is close to an isotropic shell. Fitting an isotropic Gaussian $\mathcal{N}(\mu, \sigma^2 I_d)$ to all 2,336,736 encoded Push-T frames ($d = 192$, μ and $\sigma^2 = \text{tr}(\Sigma)/d$ from the empirical mean and covariance) predicts $\|z - \mu\|$ should follow a $\chi_d(\sigma)$ distribution with mean 13.88. The true encoded frames match this almost exactly (empirical mean 13.88, figure 4), and the participation ratio $\text{tr}(\Sigma)^2/\text{tr}(\Sigma^2) = 188.5$ out of 192 dimensions (98%) confirms the covariance itself is close to isotropic, so the χ_d comparison is not being flattered by a degenerate few dominant directions.

A straight-line midpoint is far off the shell; the real trajectory is not. For 200,000 same-episode frame pairs 100 steps apart (the same-ep-100 horizon), both endpoints sit on the shell as expected (start mean 14.15, goal mean 13.60), and so does the actual logged frame at the trajectory's true midpoint (mean 13.81). The linear interpolation $\frac{1}{2}(z_{\text{start}} + z_{\text{goal}})$ does not: its mean radius is 10.03, 5.42 shell- σ below the shell mean, and 96.4% of the 200,000 interpolated midpoints fall below the shell's own 1st percentile radius (figure 2 in section 5). The two populations do not overlap. This isolates the effect to interpolation itself, not to trajectory midpoints being unusual in general: the real dynamics stay on the shell, curving around it rather than cutting through its interior, which is exactly the structure a graph built in a temporal-distance representation is meant to follow instead of a raw latent straight line (sections 3 and 4.2).

¹<https://aseemrb.me/blog/high-dimensional-gaussians-on-a-sphere/>

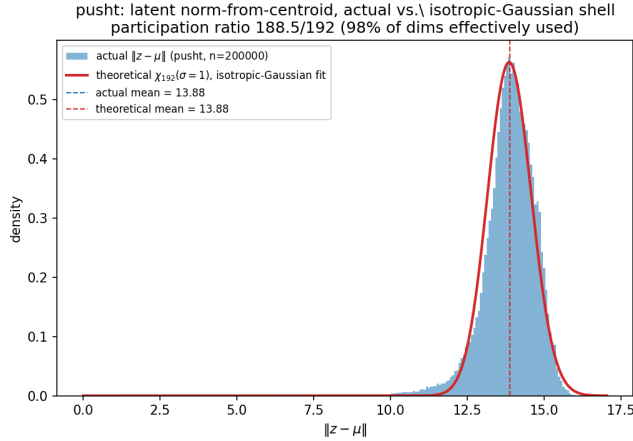


Figure 4: Push-T: actual $\|z - \mu\|$ over all 2.3M encoded frames vs. the χ_{192} prediction from an isotropic Gaussian fit to the same mean/covariance.

Scope. This diagnostic is Push-T-only and tests the naive linear interpolation, not the predictor’s own imagined rollout under the z -space-L2 objective CEM actually optimizes; whether the predictor’s rollout exhibits the same off-shell dip, and whether the effect holds on Reacher and Cube, is not yet measured. *Scripts:* `scripts/gas_mpc-gaussian-shell-diag.py` and `scripts/gas_mpc-gaussian-shell-interp-diag.py`.

E CANDIDATE RANKING WITHIN CEM’S OWN ELITES

A separate check of whether OUR’s cost ranks candidates better than z -space L2 not on a fixed diagnostic pool but on CEM’s own live search: for 25 held-out Push-T tasks per same-ep protocol (seed 0), OUR’s first CEM solve is run to iteration 1 only – the one point where both costs score the identical, as-yet-unbiased 300-candidate population sampled from CEM’s default zero-mean initialisation, before either cost’s own elite selection has shaped anything. OUR’s 30 elite candidates at that iteration are executed for real in the simulator and scored against the privileged real-physics recovery-time oracle (sections D.1 and 5’s mechanism).

Table 8: Spearman(cost, oracle recovery time) over OUR’s own 30 elite candidates at CEM iteration 1, 25 tasks. Same25/50: 25/24 tasks with non-degenerate oracle labels; same100: 11/25 (the rest were fully censored even at a widened oracle cap).

protocol	$\rho(\text{OUR})$	$\rho(\text{L2})$
same-ep 25	0.327	0.396
same-ep 50	0.203	0.086
same-ep 100	0.191	−0.064

At same-ep 25, L2 is nominally ahead, though 95% task-bootstrap confidence intervals overlap. At same-ep 50 and 100, OUR is ahead, and is the only cost of the two whose interval excludes zero at both – L2’s ranking is statistically indistinguishable from chance there, while OUR’s is not. This is the same short-range/long-range crossover table 1 shows in closed-loop success, now shown at the level of a single CEM iteration’s candidate ranking rather than a full episode. *Script:* `scripts/gas_mpc-cep-trace-audit.py`.

F PARAMETER COUNTS

Table 9 lists the parameters of the frozen LeWM stack and of the two components we train. Only ψ_0 of the TDR’s two networks is used at planning time.

Table 9: Push-T parameter counts. The LeWM encoder and predictor are frozen.

LeWM encoder (ViT-tiny)	5.50M
LeWM predictor	10.79M
LeWM action encoder	0.16M
LeWM total (frozen)	18.03M
TDR (used, ψ_0 only)	0.64M
TDR (full 2-net ensemble)	1.29M
Critic (per member)	0.83M
Critic (5-member ensemble)	4.13M

G HYPERPARAMETERS

Table 10 lists every hyperparameter shared across Push-T, Reacher, and Cube; table 11 lists the handful that are environment specific. All are argparse/Hydra defaults in `scripts/gas_mpc_prepare.py`, `scripts/viability_train.py`, `config/eval/solver/cem.yaml`, and `scripts/gas_mpc_eval.py`, run unchanged for every headline result in table 1 (Push-T: `scripts/ada_gas_mpc_phase8.sh` config B; Reacher/Cube: `scripts/ada_gas_mpc_reacher.sh` / `scripts/ada_gas_mpc_cube.sh` config B) – nothing in either table was tuned per environment beyond the two calibrated scalars in table 11.

Table 10: Hyperparameters shared across all three environments. TDR and graph follow section 3.1; critic follows section 3.3; CEM and the criterion follow equation (6) and algorithm 1.

component	hyperparameter	value
TDR ψ	architecture	3×512 LayerNorm+GELU MLP, linear head
	embedding dim	32
	expectile τ	0.99
	discount γ	0.99
	training steps	50,000
	batch size	1024
	learning rate	3×10^{-4}
Graph	TE threshold θ_{TE}	0.9
	clustering radius	$H_{TD}/2$
	goal attachment radius	$\max(H_{TD}, 1.2 d_{\min})$
Critic V	ensemble members	5
	architecture (per member)	3×512 MLP + SiLU, linear head
	max horizon $h_{\max}$	50 env steps
	horizon grid	multiples of 5, $0-h_{\max}$
	training steps	60,000
	Bellman pessimism κ	1.0
	loss weights ($\lambda_{\text{Bellman}}, \lambda_{\text{mono}}$)	1.0, 1.0
	learning rate	3×10^{-4}
	EMA target τ	0.995
CEM (LeWM, unchanged)	cross-episode negative gate	$\ \psi(t) - \psi(g)\ \geq 1.25 \times d_{\text{cal}}(h)$
	population / iterations / elites	300 / 30 / 30
	horizon K (blocks $\times$ frameskip)	$5 \times 5 = 25$ env steps
	replan interval (receding horizon)	25 env steps
Criterion (equation (6))	critic weight β	1
	critic cost	E/T (budget-capped expected hitting time)
	final-phase metric	LeWM’s own z-space L2 to the goal

Table 11: Environment-specific hyperparameters. u_k denotes the calibrated TDR distance of a k -env-step transition (median over training transitions), read from each environment’s own TDR calibration. $\lambda = u_{25}$ and $H_{\text{TD}} = u_{12}$ in every environment. Every other hyperparameter is identical across environments.

	Push-T	Reacher	Cube
H_{TD} (TDR units)	7.59	3.72	7.25
$\lambda = u_{25}$ (TDR units)	13.53	5.83	13.63